

Programação em Python

Unidade 3

Desenvolver Aplicações Back-end para Web com Python

Sessão 09

Python

Utilizando os Templates em seu APP - continuação



Python

Criando páginas para seu APP

1. Crie a pasta “*templates*” no respectivo diretório do APP
2. Agora nesta pasta, crie a pasta com o nome de seu APP. Por exemplo “tipodeatividade”
3. Cada APP terá suas páginas criadas neste diretório

```
✓ tipodeatividade
  > __pycache__
  > migrations
  ✓ templates \ tipodeatividade
```

Python

Criando páginas para seu APP - continuação

1. Criando um documento HTML para listar os tipos de atividade a partir da página de *template* principal que se encontra na pasta “*templates*” do **projeto**
2. Esta página deverá ser criada na pasta “*templates\<nome-app>*” do APP

✓ templates \ tipodeatividade

↔ atualizarTipoDeAtividade.html

↔ cadastrarTipoDeAtividade.html

↔ listarTipoDeAtividade.html

Python

Criando as URLs para o APP “Tipo de Atividade”

1. Criar o arquivo “urls.py”
2. No arquivo, criar
 1. A definição “*app_name = tipodeatividade*”. O valor de “*app_name*” será utilizado nos templates HTML
 2. Criar a lista “*urlpatterns*” com os *paths* (caminhos) para o APP

```
urls.py
1 from django.urls import path
2 from . import views
3
4 app_name = 'tipodeatividade'
5
6 urlpatterns = [
7     path('lista/', views.listar, name='listar'),
8 ]
9
10
```

Python

Criando a “view” para listar os registros da tabela “Tipo de Atividade”

1. Definir uma função “*listar*” conforme o código abaixo
2. Na função:
 - i. Recuperar os registros da tabela através de ORM
 - ii. Criar um dicionário contendo os objetos da lista. A chave/nome será utilizada na página HTML correspondente.
 - iii. Repassar o nome da página junto com o nome da pasta criada e o dicionário. Este último deverá ser atribuído ao parâmetro “*context*” da função “*render*”

```
1 from django.shortcuts import render, redirect
2 from django.http import HttpResponseRedirect
3
4 from tipodeatividade.models import TipoDeAtividade
5 from tipodeatividade.forms import TipoDeAtividadeForm, TipoDeAtividadeAtualizarForm
6 from turma.models import Turma
7
8 # Create your views here.
9 def listar(request):
10     lista_tipodeatividade = TipoDeAtividade.objects.all()
11     contexto = {
12         'tiposdeatividade': lista_tipodeatividade
13     }
14
15     return render(request, 'tipodeatividade/listarTipoDeAtividade.html', context=contexto)
16
```


Python

Criando o documento HTML

1. Criar um arquivo HTML chamado “listarTipoDeAtividade.html” para apresentar o conteúdo desejado
2. Estender o *template* principal utilizando a expressão `{% extends “<nome-do-template>.html” %}`
3. Preencher os blocos do *template* com o código HTML e Python quando for necessário
4. Utilize a instrução `{% url ‘<app-name>:<url-name>’ %}` para montar o menu e acessar o path diretamente

```
1  {% extends "base_template.html" %}
2
3  {% block title %}
4      <title>Django - Exercícios</title>
5  {% endblock %}
6
7  {% block menu %}
8      <div class="collapse navbar-collapse justify-content-end" id="mynavbar">
9          <div class="navbar-nav">
10             <a href="/" class="nav-link">Início</a>
11             <a href="{% url 'tipodeatividade:cadastro' %}" class="nav-link">Cadastrar</a>
12             <a href="{% url 'tipodeatividade:listar' %}" class="nav-link">Listar</a>
13          </div>
14      </div>
15  {% endblock %}
```

Python

Utilizando o dicionário repassado à função “render” em *listar(request)*

1. Inserir código Python com HTML para montar a tabela a ser apresentada
2. Observar que o código Python fica entre os delimitadores `{%...%}`

```
25
26  {% if tiposdeatividade %}
27
28      <!-- tabela responsiva -->
29  >  <div class="table-responsive">...
72      </div>
73
74  {% else %}
75
76  <div class="text-danger">
77      <p>Não há tipos de atividade cadastrados.</p>
78  </div>
79
80  {% endif %}
81
```

Python

Utilizando o dicionário repassado à função “render” em *listar(request)*

1. Observar como utilizar os registros da lista representada pela chave do dicionário, montando as linhas e colunas da tabela a ser apresentada

```
28     <!-- tabela responsiva -->
29     <div class="table-responsive">
30
31         <!-- classes para tabela (Bootstrap) , tabela sem borda, conteudo alinhado ao
32         <table class="table table-borderless text-center my-table-hover
33             my-table-image my-table-td-first-child my-table-border-bottom
34             my-table-th w-100">
35
36             <!-- cabecalho -->
37 >         <thead>...
45         </thead>
46
47         <!-- linhas -->
48         {% for ta in tiposdeatividade %}
49
50             <tr>
51                 <td class="text-center"> {{ ta.codigo }} </td>
52                 <td class="text-left"> {{ ta.descricao }} </td>
53             </tr>
54
55         {% endfor %}
56
57     </table>
```

Python

Formulários HTML e outros

1. Ao montar uma página, deve ser observado o fluxo de chamadas
2. Em algumas situações, a função definida na *view* correspondente, precisa acessar o banco de dados para recuperar registros para, por exemplo, montar elementos “*select*” do HTML ou até mesmo preencher formulários com os dados já gravados permitindo sua atualização.
3. No caso de páginas HTML que contenha formulários, deverá ser criado um arquivo “*forms.py*”.
4. Neste arquivo será definido o mapeamento do formulário responsável pela inserção ou atualização de registros das tabelas utilizadas no App.

Python

Trabalhado com formulários com Django

1. Criar o arquivo “*forms.py*”
2. Definir as classes a serem utilizadas utilizando herança da módulo/classe “*forms.Form*”.

```
1 from django import forms
2
3 class TipoDeAtividadeForm(forms.Form):
4     descricao = forms.CharField(max_length=100, required=False, help_text='Informe a descrição do Tipo de Atividade')
5
```

Obs.:

- Os valores do atributo “*name*” dos elementos de formulários HTML devem, preferencialmente, serem iguais aos atributos/propriedades da classe do form.
- Cada atributo definido é uma instância de um campo, por exemplo, *forms.CharField* ou *forms.DateField*.
- O Django utiliza *metaclasses* para transformar os atributos em uma estrutura interna, assim, será possível, criar widgets HTML, validar dados enviados e armazenar valores limpos.
- Ao instanciar um formulário, o Django associa os dados enviados pelo método *Post* do HTML aos campos, executa validações e prepara mensagens de erro, se houver.

Python

Trabalhando com formulários com Django

1. Criar uma *view* para atender à chamada de uma URL para devolver uma página com um formulário HTML que servirá com entrada de dados

```
23 def carregar_cadastro(request):  
24     return render(request, 'tipodeatividade/cadastrarTipoDeAtividade.html')  
25
```

Python

Trabalhando com formulários com Django

1. Criar uma página HTML com um formulário para entrada de dados
2. Após a definição da abertura do elemento “*form*” (<*form*...>) do HTML, insira o comando de linguagem de *templates* do Django:

```
{% csrf_token %}
```

Atenção:

- As finalidades de comando são:
 - i. Criar um token único por sessão ou requisição.
 - ii. Associar um valor único e imprevisível à sessão do usuário que está chamando o site.
 - iii. Validar no servidor se o token recebido está de acordo com o esperado.
 - iv. Prevenir que requisições forjadas de outros sites sejam aceitas, trazendo segurança para a aplicação
- O “**CSRF**” de “*csrf_token*” representa a sigla de um tipo de ataque malicioso, ou seja, um ataque **CSRF – Cross-Site Request Forgery**.
- Este ataque acontece quando um site malicioso induz o navegador de um usuário autenticado a enviar requisições indesejadas para outro site no qual ele já está logado, explorando a sessão ativa.

Python

Trabalhando com formulários com Django

```
17 {% block corpo %}
18 <main class="container m-2 w-75 mx-auto my-font-family">
19
20 <div class="row">
21 <div class="col text-center m-0">
22 <h2 class="mt-2 mb-5 my-h2">Cadastro de Tipo de Atividade</h2>
23 </div>
24 </div>
25
26 <form action="{% url 'tipodeatividade:cadastrar' %}" method="POST"
27 name="frmCadastrarTipoDeAtividade" id="idFrmCadastrarTipoDeAtividade">
28 {% csrf_token %}
29 <div class="form-group">
30 <label for='idDescricao'>Descrição</label>
31 <input class="form-control" type='text' id='idDescricao' name='descricao'
32 placeholder='Informe a descrição' required>
33 </div>
34 <div class='form-group text-center'>
35 <input type='reset' value='Limpar' class='btn btn-primary btn-md'>
36 <input type='submit' value='Cadastrar' class='btn btn-primary btn-md'
37 id="btnCadastrarTipoDeAtividade">
38 </div>
39 </form>
40 </main>
41 {% endblock %}
```


Python

Trabalhando com formulários com Django

1. Criar uma view para tratar o formulário e registrar os dados no banco de dados através de ORM

```
27 def cadastrar(request):
28     form = TipoDeAtividadeForm(request.POST)
29     if form.is_valid():
30         dados_tipodeatividade = form.cleaned_data
31         tipodeatividade = TipoDeAtividade(
32             descricao = dados_tipodeatividade['descricao']
33         )
34
35         tipodeatividade.save()
36
37     return render(request, 'tipodeatividade/cadastrarTipoDeAtividade.html')
```

Python

Trabalhando com formulários com Django

1. Utilizar o atributo `form.cleaned_data`, que retornará um dicionário que contém os dados do formulário já validados e convertidos para os tipos corretos.
2. As chaves do dicionário são os nomes dos campos do formulário, sendo definidos, já, com seu tipo de dado correto.
3. Atenção:
 - O dicionário ficará disponível depois que o método `form.is_valid()` for executado.
 - Este método valida os dados do formulário enviado através do *request*

Python

Trabalhando com formulários com Django

1. É possível criar validações próprias na classe que representa o formulário

```
10 class Aluno(forms.Form):
11     nome = forms.CharField()
12     idade = forms.IntegerField()
13
14     def clean(self):
15         dados = super().clean()
16         idade = dados.get('idade')
17
18         if idade is not None and idade < 18:
19             raise forms.ValidationError("Idade mínima é 18 anos.")
20
21     return dados
```

Python

- Considerações finais.
- Agradecimentos.



Programação em Python

Botafogo



Até a próxima!

